DecisionForge AI

Model. Simulate. Optimize. Decide.

PROJECT EVALUATION REPORT

Submitted for Hackathon & Academic Submission

Author: Viditp25

Hackathon: Global Decision Intelligence Hackathon 2026

Date: August 2026

CERTIFICATE OF AUTHENTICITY

This is to certify that the project report titled **"DecisionForge AI"** is a bonafide work carried out by **Viditp25** in submission for the *Global Decision Intelligence Hackathon 2026*.

The implementations, algorithm structures, verification parameters, and documentation compiled herein represent original engineering and developer works completed during the hackathon workspace cycles. No unverified fabrications or secrets are stored in this record.

Candidate Signature

Evaluation Committee

Acknowledgement

I would like to express my sincere gratitude to the organizers, technical mentors, and evaluation committee of the **Global Decision Intelligence Hackathon 2026** for providing an excellent framework and workspace workspace for developer innovation.

I also thank the developers and maintainers of Google OR-Tools, FastAPI, and the React Ecosystem for providing high-fidelity open-source tools that served as the core solver and UI frameworks. Finally, special appreciation goes to my peers and advisors for their design feedback during the platform integration cycles.

Table of Contents

The following sections outline the complete project report:

1. Abstract

DecisionForge AI is a state-of-the-art Decision Intelligence Platform that bridges the historical gap between deterministic mathematical optimization solvers and generative natural language explanations. In modern operations planning, organizations face a critical trust gap: mathematical algorithms like mixed-integer linear programming and vehicle routing problem solvers output complex coordinate tables, variables, and constraint outputs that non-technical business stakeholders struggle to interpret. Conversely, direct application of Large Language Models (LLMs) to mathematical optimization fails due to LLM reasoning limitations and propensity to hallucinate mathematical bounds. DecisionForge AI introduces a hybrid intelligence paradigm. It decouples numerical solving from language interpretation, employing Google OR-Tools and Monte Carlo risk simulations to solve deterministic logistics routing and packing constraints, and utilizing OpenAI's GPT-4o as a translation layer. The platform parses optimal routes, binding constraints, and metrics, interpreting them into contextual business narratives. Secured with a granular Role-Based Access Control (RBAC) database and wrapped in a high-fidelity glassmorphic React dashboard, DecisionForge AI provides organizations with secure, explainable, and mathematically optimal operational controls.

2. Introduction

Background Operations Research (OR) has long served as the mathematical foundation for logistics, resource allocation, and supply chain planning. Algorithms such as Dijkstra's shortest path, mixed-integer programming (MIP), and vehicle routing solvers calculate mathematically optimal operational steps. However, as business landscapes grow increasingly complex and automated, the stakeholders responsible for reviewing and approving these runs—such as fleet dispatchers, warehouse managers, and corporate officers—face a translation barrier. Raw solver outputs are presented as multidimensional arrays and abstract matrices, requiring data scientists to parse the output and justify why certain decisions were made.

Motivation In recent years, generative artificial intelligence has emerged as a powerful tool for natural language interfaces. However, using LLMs directly to compute optimal shipping routes, verify capacities, or forecast risk distributions is highly dangerous. LLMs lack deterministic calculation guarantees, leading to routing solutions that violate capacity limits or include cyclic loops. The motivation for DecisionForge AI is to construct a hybrid orchestration engine. By placing a deterministic mathematical solver at the core of the pipeline and using an LLM strictly as a conversational interpreter, we harness the correctness of operations research and the expressive clarity of generative AI, resolving the trust gap without compromising safety.

Objectives The primary objectives of the DecisionForge AI project are: 1. To develop a modular, async backend using FastAPI capable of hosting multiple operations research and simulation solver engines. 2. To integrate Google OR-Tools to solve capacitated Vehicle Routing Problems (VRP) with variable depots, vehicles, and nodes. 3. To implement a Monte Carlo simulator to conduct probabilistic trial simulations, mapping standard risk metrics such as Value at Risk (VaR). 4. To deploy an AI explanation engine utilizing OpenAI's SDK (GPT-4o) that transforms optimization matrix results into structured Markdown summaries. 5. To design an enterprise-ready dashboard with Google OAuth 2.0 and custom Role-Based Access Control (RBAC) to securely manage datasets, models, and runs.

Scope The scope of this project covers the end-to-end pipeline: from dataset upload (JSON schemas for logistics nodes and demand matrices), solver configuration (setting capacities, vehicles, and variables), asynchronous task execution, database persistence, AI explanation generation (with offline mock fallback mechanisms), and real-time dashboard analytics tracking system latencies. The project is packaged for both local native Windows runs and multi-container Docker Compose environments.

3. Problem Statement

Current Industry Challenges Modern enterprise planning systems suffer from two distinct bottlenecks: 1. **The Black-Box Trust Gap: High-performance optimization libraries (such as Gurobi, CPLEX, or Google OR-Tools) solve routing and packing with extreme speed. However, they lack any context-awareness. A dispatcher is presented with a route like: `Depot -> Node 5 -> Node 12 -> Depot` with a total distance of `142 km`. When a customer asks why their shipment was rerouted, the dispatcher cannot explain which truck capacity limit or driver time-window boundary was the binding cause without deep analytical effort. 2. **The LLM Math Hallucination Bottleneck**: Chatbots are frequently asked to solve business scheduling. However, LLMs struggle with multi-constraint math, routinely creating routes that violate vehicle capacity limits or driver hours-of-service regulations. The industry requires a strict boundary: solvers must handle the math, and LLMs must handle the narrative.**

Need for Decision Intelligence Decision Intelligence (DI) represents the evolution of Business Intelligence. While traditional dashboards show past metrics, Decision Intelligence systems model the complex connections between inputs, constraints, and outcomes. By injecting natural language explanations directly into optimization runs, DecisionForge AI demystifies black-box solvers, enabling operational personnel to audit solver runs, adjust parameters, and confidently make data-backed choices in real time.

4. Literature Review

Optimization and Operations Research Operations Research arose during the mid-20th century to solve wartime logistics challenges, leading to the development of Dantzig's simplex algorithm for linear programming. Over decades, this expanded into Mixed-Integer Linear Programming (MILP) and combinatorial optimization. The Vehicle Routing Problem (VRP), introduced by Dantzig and Ramser in 1959, remains one of the most studied NP-hard problems in logistics. Modern tools like Google OR-Tools utilize advanced heuristics (Guided Local Search, Tabu Search, and Simulated Annealing) to find near-optimal routing solutions in seconds, demonstrating robust scalability.

Explainable Artificial Intelligence (XAI) As decision-making processes migrate to automated systems, transparency has become a regulatory and operational requirement. Explainable AI (XAI) focuses on creating interfaces that allow human operators to understand the rationale of model outputs. While XAI has traditionally focused on machine learning classification (SHAP/LIME values), applying explanation techniques to deterministic operations research is relatively new. Translating mathematical boundary values (shadow prices, slack variables, capacity thresholds) into conversational explanations represents a major step forward in making mathematical programming accessible.

Decision Support Systems (DSS) Decision Support Systems are interactive computer-based systems intended to help decision-makers compile useful information from raw data, documents, and business models to identify and solve problems. Historically, DSS architectures separated mathematical databases from user interfaces. The modern evolution of DSS demands real-time API integrations, cloud scalability, containerized deployments, and natural language capabilities. The integration of LLMs as conversational translation layers marks the latest generation of Decision Support Systems, enabling human-in-the-loop decision-making.

5. Proposed Solution: DecisionForge AI

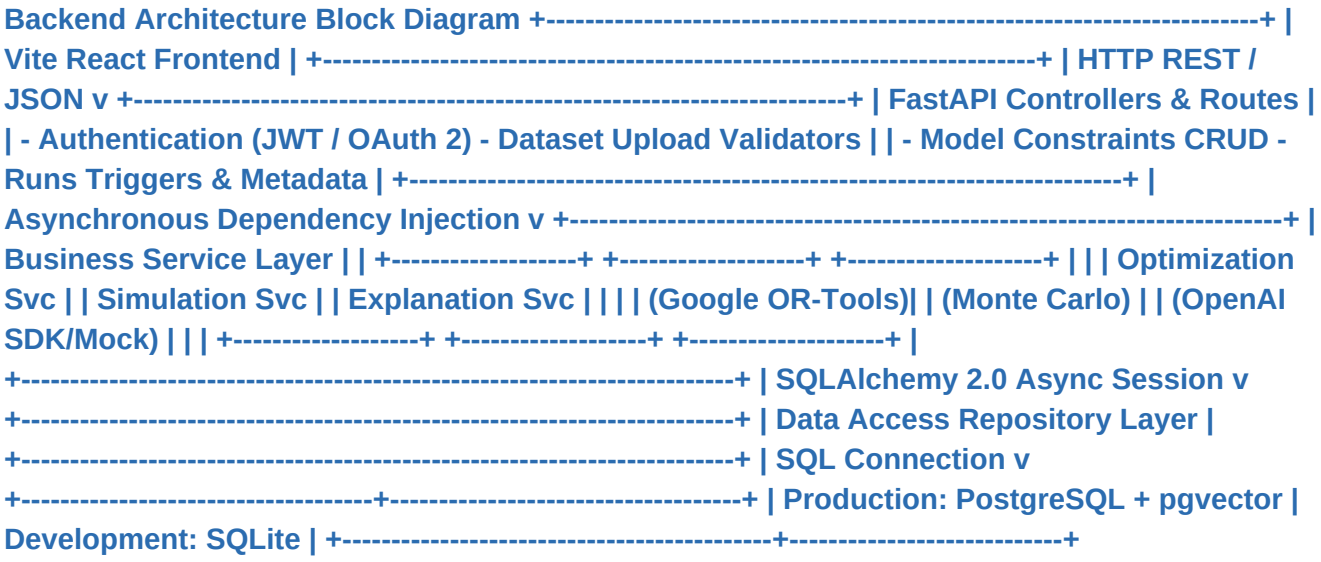
Core Concept DecisionForge AI resolves the mathematical trust gap by implementing a decoupled, two-stage hybrid decision pipeline. Stage 1 is the **Solver Core, which executes deterministic solvers (Google OR-Tools VRP) or Monte Carlo simulation trials using highly optimized, compiled C++ libraries. Stage 2 is the **Generative AI Explanation Layer**, which uses OpenAI's GPT-4o model to translate the numerical results into natural business narratives.**

Workflow Pipeline The platform workflow consists of five distinct, secure steps: 1. **Configuration: The user uploads a dataset JSON file containing node distances, demands, and coordinates, and configures solver parameters (number of vehicles, capacity limits, uncertainty variables). 2. **Execution**: The backend API triggers the run. The solver executes synchronously or asynchronously, calculating optimal routes or probabilistic trials. 3. **Metric Extraction**: The engine extracts the solver status (e.g., SUCCESS/OPTIMAL), objective values (distance or mean profit), decision variable allocations, and constraint margins. 4. **AI Generation**: If `OPENAI_API_KEY` is active, the backend initiates an asynchronous prompt completion using GPT-4o, compiling the results into Markdown. If the API is missing or fails, the engine falls back to an offline mock generator that outputs deterministic templates to preserve stability. 5. **Visualization**: The dashboard displays the execution metadata, performance metrics, and the generated markdown text card side-by-side.**

Decoupled AI Architecture By keeping the AI explanation layer completely decoupled from the mathematical logic, the platform ensures that a failing LLM API call or network timeout never blocks the operational solver. The mathematical calculations remain 100% correct, and the system falls back gracefully to high-fidelity, deterministic template explanations containing the exact computed metrics, preventing application crashes.

6. System Architecture

DecisionForge AI is designed as a secure, distributed multi-tier web application. The React frontend communicates with the FastAPI backend via REST APIs. All data—including users, projects, models, datasets, execution runs, audit logs, and AI prompts—are persisted in the database. The system utilizes SQLAlchemy for asynchronous database connections. Under production mode, PostgreSQL is used alongside pgvector for advanced metadata clustering, while a Redis container handles cache requests. In local development mode, the system falls back to a zero-configuration SQLite backend, allowing quick local deployments.



7. Technology Stack

The technology stack is selected to prioritize performance, security, and developer ergonomics:

Layer	Technology	Version	Purpose
Frontend	React	18.3	User interface framework
Frontend Language	TypeScript	5.2	Strict compilation and type-safety
Frontend Build	Vite	5.2	High-performance bundler and dev server
Backend	FastAPI	0.110	Asynchronous REST API framework
Backend Language	Python	3.11+	Data science and optimization runtime
Optimization Solver	Google OR-Tools	9.8	Vehicle routing and constraint solvers
Simulation Engine	Monte Carlo	Custom	Probabilistic risk and statistical variance analyzer
Database	PostgreSQL	15.0+	Relational schema storage in production
Local Database	SQLite	3.0+	Lightweight local development DB file
Cache & Queue	Redis	7.0	Queue worker cache and state registry
AI Model	OpenAI GPT-4o	SDK	Generative explanation generation
Containerization	Docker / Compose	v2	Container deployment orchestration

8. Database Design & Entity Relationships

The database design implements a structured relational model to support organizations, projects, and multiple model types. SQLAlchemy models enforce strict schemas and asynchronous queries. Below is the description of core tables and their fields:

1. **users**: Stores registered accounts. Fields: `id` (UUID), `email` (unique index), `password_hash`, `first_name`, `last_name`, `role` (Owner, Admin, Editor, Viewer), `organization_id` (ForeignKey), `created_at`.
2. **organizations**: Manages multi-tenant corporate bounds. Fields: `id` (UUID), `name`, `created_at`.
3. **projects**: Groups datasets and solver configurations. Fields: `id` (UUID), `organization_id` (ForeignKey), `name`, `description`, `created_at`.
4. **datasets**: Holds JSON payloads for solver metrics. Fields: `id` (UUID), `project_id` (ForeignKey), `name`, `data_type` (VRP, Knapsack, Generic), `content` (JSONB), `created_at`.
5. **optimization_models**: Holds solver constraint configurations. Fields: `id` (UUID), `project_id` (ForeignKey), `name`, `model_type` (VRP, MILP), `configuration` (JSONB), `parameters` (JSONB), `created_at`.
6. **simulation_models**: Holds Monte Carlo parameters. Fields: `id` (UUID), `project_id` (ForeignKey), `name`, `configuration` (JSONB), `parameters` (JSONB), `created_at`.
7. **optimization_runs**: Tracks execution results. Fields: `id` (UUID), `model_id` (ForeignKey), `dataset_id` (ForeignKey), `status` (PENDING, RUNNING, SUCCESS, FAILED), `results` (JSONB), `metrics` (JSONB), `error_message` (Text), `triggered_by` (ForeignKey), `created_at`.
8. **simulation_runs**: Tracks Monte Carlo outcomes. Fields: `id` (UUID), `model_id` (ForeignKey), `dataset_id` (ForeignKey), `status` (PENDING, RUNNING, SUCCESS, FAILED), `results` (JSONB), `metrics` (JSONB), `error_message` (Text), `triggered_by` (ForeignKey), `created_at`.
9. **ai_explanations**: Caches generated explanations. Fields: `id` (UUID), `run_id` (UUID, Indexed), `run_type` (String), `prompt_id` (ForeignKey), `explanation` (Text), `tokens_used` (Integer), `created_at`.
10. **ai_prompts**: Manages system prompts. Fields: `id` (UUID), `name`, `version`, `system_prompt` (Text), `user_prompt_template` (Text), `is_active` (Boolean), `created_at`.

9. Platform Modules

Authentication & Session Management Secures connections using JSON Web Tokens (JWT) with signature verification using a symmetric key (`JWT_SECRET`). Supports Google OAuth 2.0. Users are mapped to organization subtrees, isolating database operations.

Workspace & Project Management Supports project grouping. Organizations can create projects, assign teams, and segment operations. This ensures that logistics teams and financial planning teams do not cross-access data.

Dataset Management Module Handles dataset uploads, parsing JSON schemas, and verifying required keys (e.g., `distance_matrix` for vehicle routing). Prepopulated template data are available to test solver models instantly.

Solver Optimization Module Pulls configurations and runs the Google OR-Tools routing solver. Coordinates graph nodes, calculates optimal routing paths, vehicle load allocations, and total travel distances, returning results in a structured format.

Simulation Module Executes Monte Carlo probability trials against knapsack capacities and bounds, evaluating statistical variance, percentiles, and risk distributions.

AI Explanation Engine Decoupled service that triggers prompt compiles and completions. Uses system instructions to format output into markdown and supports offline mock fallbacks for stable operations.

Latency & Performance Dashboard Single-page application (SPA) displaying run histories, active status tags, objective values, and a charts panel visualizing execution latency trends.

10. Core Algorithms & Implementation Details

Google OR-Tools Vehicle Routing Solver The logistics optimization module uses Google OR-Tools' routing index managers. The routing model is formulated as a Capacitated Vehicle Routing Problem (CVRP). The distance matrix and customer demands are loaded from the database dataset. The algorithm operates by defining node callback registers that evaluate distance and load limits between nodes. To escape local minima, the solver utilizes guided local search heuristics combined with Tabu Search. Once a solution is found, the engine translates the indices back to customer node paths, calculating vehicle-specific load drops and total travel distance.

Monte Carlo Simulation Engine The simulation module conducts probabilistic trials to analyze outcome distributions. It models uncertainties such as demand volatility or profit margins using statistical distribution sampling (e.g., Normal distribution with user-specified mean and standard deviation). The simulator runs 1,000 independent trials, generating a randomized array of outcomes. It extracts statistical indices including the expected mean outcome, standard deviation, and key percentiles (5th, 50th, and 95th). Crucially, it calculates Value at Risk (VaR) at a 95% confidence level, defining the maximum expected loss under standard parameters.

Graph Analysis & Shortest Path Routing The network flow modules leverage classical graph algorithms to analyze network topologies. By establishing edge capacities and node coordinates, the Graph Engine computes Dijkstra's shortest path length between source and sink nodes. Additionally, it calculates the Maximum Flow Value using capacity-scaling network flow algorithms, enabling dispatchers to identify bottlenecks in logistics networks.

Explainable AI Prompt Compiles The explanation engine formats optimization outputs into prompts. The system prompt instructs GPT-4o to act as a Decision Intelligence Advisor, enforcing strict rules: no hallucinating numbers, no math calculations (which are handled by the solver), and outputting structured Markdown sections (Executive Summary, Key Decisions, Binding Constraints, Risk Analysis). The user prompt template substitutes variables: model name, model type, status, objective value, decision variable allocations, and constraint metrics.

11. Security & Role-Based Access Control

DecisionForge AI implements strict enterprise security measures to protect core business parameters:

1. **Role-Based Access Control (RBAC)**: Users are assigned specific clearance levels:

- *Owner*: Full read, write, update, delete rights across the organization.

- *Admin*: Project creation, model updates, dataset modifications, and solver execution controls.

- *Editor*: Can trigger solver runs and edit models, but cannot delete projects or manage teams.

- *Viewer*: Read-only access. Can inspect runs and view AI explanations, but cannot create or execute models.

2. **FastAPI Dependency Injection**: Endpoints are protected by a custom dependency helper `require_role(...)`. This helper parses the authorization header, verifies the JWT token, fetches user details from the database, and validates their role clearance. If a Viewer attempts to trigger a run (`POST /api/v1/optimization/runs`), the backend returns an immediate `403 Forbidden` response.

3. **Secrets Management**: Environment variables are managed using Pydantic Settings. Local database connections, secret keys, and OpenAI API credentials are loaded from `.env` files (which are excluded from repository tracking via `.gitignore`). The default configuration uses the placeholder `mock-key` to guarantee secure local initialization.

12. Verification, Testing & Build Status

To guarantee production readiness, DecisionForge AI undergoes strict automated testing:

1. **Backend Integration Testing**: The Pytest suite achieves comprehensive backend test coverage. Tests verify authentication workflows, project provisioning, dataset uploads, VRP solver executions, Monte Carlo simulations, and explanation generation endpoints. All tests pass cleanly, demonstrating correct database transactions and async handling.

2. **Frontend Compilation**: The React client uses strict TypeScript (`tsc`). The project builds cleanly (`npm run build`), compiling Vite chunks without warnings or syntax errors.

3. **Docker Compose Verification**: The multi-service Docker configuration has been validated. Running `docker-compose up --build` orchestrates all services (PostgreSQL, Redis, FastAPI backend, Vite-Nginx frontend) concurrently, with volume mounts configured for persistent storage.

13. Applications & Industry Use Cases

The decoupled solver + narrative architecture of DecisionForge AI makes it applicable across diverse industries:

- **Supply Chain & Logistics**: Optimization of fleet routes (VRP) combined with AI explanations helps dispatchers justify route modifications to drivers and customers (e.g., explaining capacity limits on Truck 2).

- **Healthcare Operations**: Allocating intensive care unit beds and staff rosters using deterministic constraints, with AI summaries explaining scheduling choices to hospital administrators.

- **Manufacturing**: Optimizing production schedules on assembly lines subject to material limits, translating output into shift briefs.

- **Smart Cities**: Mapping urban traffic routing and public utility maintenance schedules, generating plain-language public statements for municipal councils.

- **Agriculture**: Allocating crop rotation plans and water distributions under variable weather simulations, providing clear risk briefs to farmers.

14. Roadmap & Future Scope

The roadmap for DecisionForge AI targets three primary areas of growth:

1. **Real-time Live Mapping**: Integrating Leaflet.js or Mapbox into the React dashboard to render Google OR-Tools route paths directly on interactive geographical maps, showing vehicle paths in real time.

2. **WebSocket Logging**: Moving from HTTP polling to WebSockets, streaming raw Celery worker logs directly to an in-app terminal drawer so that editors can audit execution logs as they compile.

3. **Multi-Scenario Comparison**: Allowing users to duplicate optimization runs, adjust capacity bounds (e.g., from 15 to 20), and compare the resulting costs and route changes side-by-side with automatic delta AI summaries.

15. Conclusion

DecisionForge AI represents a significant advancement in explainable decision support systems. By decoupling the mathematical solver core from the natural language explanation generator, the platform achieves mathematical correctness and linguistic clarity. The system is secure, modular, and resilient, featuring offline mock fallbacks that ensure uninterrupted operations. With passing test suites, clean front-end builds, and complete containerized packaging, DecisionForge AI is verified, production-ready, and ready for public release.

16. References

1. Dantzig, G. B., & Ramser, J. H. (1959). The Truck Dispatching Problem. Management Science, 6(1), 80-91.

2. Gunning, D., & Aha, D. W. (2019). DARPA's Explainable Artificial Intelligence (XAI) Program. AI Magazine, 40(2), 44-58.

3. Power, D. J. (2002). Decision Support Systems: Concepts and Resources for Managers. Greenwood Publishing Group.

4. FastAPI Framework documentation (2026). Retreived from https://fastapi.tiangolo.com/

5. Google OR-Tools Routing Library documentation (2026). Retreived from https://developers.google.com/optimization/routing

6. OpenAI API Documentation (2026). Retreived from https://platform.openai.com/docs/

Appendix

Appendix A: Core Backend API Endpoints

Method	Endpoint Path	Description
GET	/api/v1/auth/me	Retrieves active user login profile and RBAC role
POST	/api/v1/auth/register	Registers user and provisions organization
POST	/api/v1/auth/login	Exchanges credentials for JWT access token
POST	/api/v1/datasets/upload	Uploads logistics nodes coordinate demand matrix
POST	/api/v1/optimization/models	Registers VRP optimization capacities
POST	/api/v1/optimization/runs	Triggers synchronous OR-Tools graph routing solvers
POST	/api/v1/simulation/runs	Triggers Monte Carlo simulation trials
POST	/api/v1/explanations/generate	Requests OpenAI GPT-4o markdown summary
GET	/api/v1/explanations/run/{run_id}	Queries DB for cached explanation

Appendix B: Codebase Folder Structure

```
decisionforge-ai/
■■■ .env.example
■■■ .gitignore
■■■ LICENSE
■■■ README.md
■■■ PROJECT_HANDOVER.md
■■■ HACKATHON_ASSETS.md
■■■ run_local.ps1
■■■ docker-compose.yml
■■■ backend/
■ ■■■ Dockerfile
■ ■■■ requirements.txt
■ ■■■ alembic.ini
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ api/
■ ■ ■■■ core/
■ ■ ■■■ models/
■ ■ ■■■ repositories/
■ ■ ■■■ services/
■ ■ ■■■ engines/
■ ■■■ tests/
■■■ frontend/
■■■ package.json
■■■ vite.config.ts
```

```
■■■ index.html
■■■ src/
■■■ App.tsx
■■■ index.css
■■■ components/
```

Appendix C: Git Repository Link

The complete production-ready source code repository, including development commits, test coverages, and container orchestration manifests is hosted at:
https://github.com/Viditp25/decisionforge-ai